SentinelLite

A Lightweight Authentication Log Analyzer and Integrity Checker

Project Report
Prepared as an undergraduate cybersecurity project

Abstract

SentinelLite is a small defensive security application that analyzes authentication log records and highlights repeated failed login attempts. The system parses a consistent text format, groups failures by source IP address, checks a configurable time window and threshold, and presents potential incidents in a dashboard. It also calculates a SHA-256 digest of the uploaded file to support integrity comparison. The project is intended for learning and demonstration using synthetic or authorized logs.

1. Introduction

Authentication logs record sign-in events and are useful for identifying unusual activity. Repeated failed attempts may indicate a password-guessing attempt, but can also result from a user mistyping a password or an application issue. SentinelLite provides a simple way to review events and investigate patterns. It does not automatically block users or claim that an alert proves an attack.

2. Problem Statement

Manual review of many authentication records can be slow and may make suspicious patterns difficult to notice. A lightweight tool is needed to parse events consistently, apply a basic detection rule, summarize findings, and provide a reproducible integrity digest.

3. Objectives

• Read authentication log files in a defined format.
• Extract timestamps, IP addresses, usernames, and status values.
• Detect a configurable number of failures from one IP within a time window.
• Display event totals and alerts in a dashboard.
• Export a concise incident summary.
• Calculate SHA-256 to help compare file integrity.

4. Tools and Technologies

Technology	Use
Python	Parsing, detection logic, report generation
Pandas	Tabular log handling and aggregation
Streamlit	Interactive dashboard
SHA-256 (hashlib)	File digest for integrity comparison

5. System Design

Input log file → parser and validation → failed-event selection → sliding-window detection → dashboard and report. The application also hashes the exact uploaded bytes with SHA-256. Invalid lines are reported rather than silently treated as valid events.

6. Detection Method

The application sorts failed events by timestamp for each IP address. It maintains a sliding window whose width is set by the user. When the number of events in the window reaches the configured threshold, the source is flagged as HIGH severity. The implementation emits at most one alert per IP during a run to reduce duplicate alerts. This is a simple educational heuristic and can produce false positives or miss activity spread across sources.

7. Input Format

```
YYYY-MM-DD HH:MM:SS,IP_ADDRESS,USERNAME,SUCCESS_OR_FAILED
2026-11-04 10:10:00,192.168.1.105,alice,FAILED
```

8. Testing Plan

• Valid successful-login event: parsed and counted.
• Below-threshold failures: no alert expected.
• Threshold number of failures within the window: alert expected.
• Failures spread beyond the window: no alert expected for that window.
• Malformed line or invalid timestamp: reported as invalid.
• File modified after digest calculation: recalculated SHA-256 should differ, except for the negligible possibility of a hash collision.

9. Limitations and Future Enhancements

Current limitations include a single input format, local file upload, simple IP-based detection, and no authentication or persistent database. Future work could include support for Linux/Windows log formats, multiple detection rules, geolocation enrichment from approved sources, role-based access, scheduled analysis, and a more robust alert lifecycle. These enhancements should be validated with authorized datasets.

10. Conclusion

SentinelLite demonstrates core defensive cybersecurity workflow: collecting a log file, parsing events, applying a detection rule, reviewing alerts, exporting a report, and checking file integrity. It is suitable as a foundation for learning security monitoring and log analysis.

References

1. Python documentation — hashlib: https://docs.python.org/3/library/hashlib.html
2. Streamlit documentation: https://docs.streamlit.io/
3. Pandas documentation: https://pandas.pydata.org/docs/

Safety note: Use synthetic data or authentication logs that you own or are explicitly authorized to analyze. Do not publish sensitive log records.